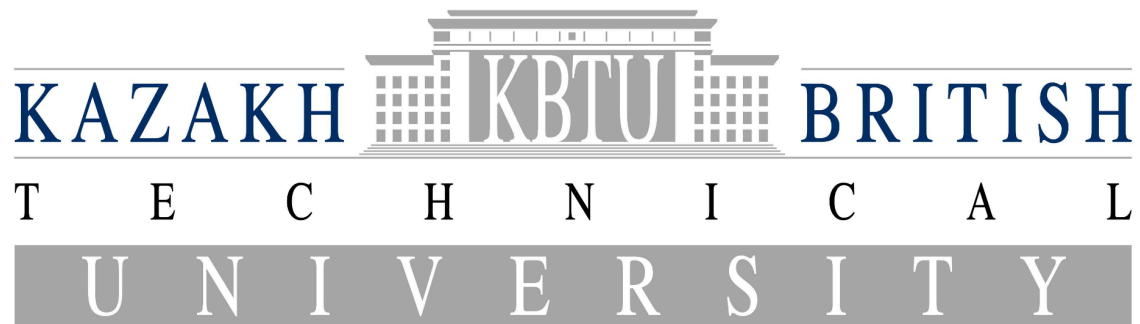




REPORT
CLOUD COMPUTING
Midterm Project

Done by: Tolegen Ernazar
24MD0491
27.10.2024

Table of contents

Executive Summary	3
Introduction	4
Project Objectives	5
Cloud Computing Overview	6
Google Cloud Platform: Core Services	7
Virtual Machines in Google Cloud	8
Storage Solutions in Google Cloud	11
Networking in Google Cloud	13
Identity and Security Management	15
Testing and Quality Assurance	17
Monitoring and Maintenance	18
Challenges and Solutions	19
Conclusion	20
References	21
Appendices	22

Executive Summary

This project focused on creating a file-sharing application using Google Cloud Platform (GCP) to provide a reliable and secure environment for storing and managing user files. The main tasks included setting up a virtual machine, configuring cloud storage, managing networking, and implementing strict security controls.

The backend of the application was deployed on a Google Compute Engine Virtual Machine (VM), ensuring a stable and customizable environment for running the server. Google Cloud Storage was utilized for file management, allowing users to upload, download, and manage files efficiently while benefiting from GCP's secure and scalable infrastructure.

Networking was configured using a Virtual Private Cloud (VPC), which allowed for isolated and secure communication between different parts of the system. Firewall rules were implemented to manage traffic, ensuring that only authorized requests could access the application's resources. Additionally, Identity and Access Management (IAM) was set up to enforce clear user roles and permissions, protecting sensitive data and maintaining the integrity of the system.

The project successfully created a secure and manageable cloud-based application, utilizing GCP's core services to provide a scalable solution for file-sharing. This experience highlighted the power of cloud platforms in building secure applications. Looking ahead, I'm excited to continue working with Google Cloud Platform. I feel more confident in managing cloud infrastructure and understand the potential it has for handling both small-scale and large-scale projects. One of the services I'm very interested in exploring further is **BigQuery** and **Data Studio**.

Introduction

Cloud computing is a technology model that provides access to computing resources—like servers, storage, and software—over the internet. This approach allows businesses to avoid the complexities of managing physical infrastructure, making development faster and more flexible. In modern application development, cloud computing is significant because it supports rapid deployment, scalability, and reliable performance.

Google Cloud Platform (GCP) was chosen for this project because of its robust service offerings and scalability. GCP's comprehensive tools for computing, storage, and networking make it easy to integrate different services seamlessly. Additionally, GCP is known for its strong security features, including encryption and Identity and Access Management (IAM), which ensure data protection. Its cost-efficient, pay-as-you-go pricing model allows the project to manage resources effectively without high initial costs. GCP's user-friendly environment, extensive documentation, and developer tools further enhance its appeal for this project.

Project Objectives

The objective of this project is to create a reliable and secure file-sharing application using Google Cloud Platform (GCP). The application aims to provide users with a simple way to upload, store, manage, and download files efficiently. This will be achieved by leveraging GCP's infrastructure to handle the backend, allowing for easy scalability and minimizing the need for manual server management.

A primary focus is on ensuring secure file storage, utilizing Google Cloud Storage to protect user data with encryption and strict access controls. The app is designed to handle a large number of users, particularly during peak periods, by taking advantage of GCP's auto scaling and load balancing features.

Networking and security will be carefully managed through Google's Virtual Private Cloud, with best practices for security enforced using Identity and Access Management to control permissions. Cost and performance optimization will be a key goal, using GCP's monitoring tools to keep the application efficient without overspending. Finally, a user-friendly interface will be developed to ensure a seamless experience for uploading, downloading, and managing files.

Cloud Computing Overview

Cloud computing is a technology model that allows access to computing resources—such as servers, storage, databases, and applications—over the internet, instead of relying on local infrastructure. This model emphasizes flexibility, scalability, and cost-efficiency, enabling organizations to reduce upfront investments and focus on development without managing physical hardware. At its core, cloud computing is driven by principles like on-demand self-service, broad network access, and resource pooling, allowing multiple users to share resources efficiently. It also offers rapid elasticity to scale resources as needed and measured service to track usage and costs transparently.

The key benefits of cloud computing include the ability to easily scale resources to handle fluctuations in demand, making it suitable for high-traffic scenarios. It reduces costs by adopting a pay-as-you-go model, where users are only charged for what they use, eliminating the need for expensive hardware purchases. The cloud also offers flexibility with a vast range of tools and services, supporting rapid development and deployment of applications. Additionally, cloud platforms are known for high availability, with built-in redundancy that ensures reliability, and robust security measures like encryption and compliance standards that protect data.

Cloud computing is structured around three main deployment models: Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS). To see their differences, see Appendix A. IaaS offers virtualized computing resources, giving users control over infrastructure components like virtual machines and storage. It's ideal for organizations seeking control over hardware and networking without managing physical servers. PaaS, on the other hand, provides a platform that allows developers to build and deploy applications without managing the underlying infrastructure, simplifying the development process. SaaS delivers fully functional software applications over the internet, accessible via a browser, relieving users from concerns about maintenance or updates. These models provide varying degrees of control and flexibility, catering to diverse needs, from full infrastructure management to simple software use.

Google Cloud Platform: Core Services

Google Cloud Platform (GCP) offers a wide range of cloud services that cater to various aspects of application development, deployment, and management. For this project, several core services are particularly relevant, as they provide the necessary tools to build a scalable and secure file-sharing application.

GCP's main services fall into categories like computing, storage, networking, and security. In terms of computing, Google Compute Engine provides virtual machines that can be customized to meet specific performance needs, while Google App Engine offers a platform-as-a-service (PaaS) environment for web applications. For storage, Google Cloud Storage enables scalable object storage that is ideal for managing and storing large files securely. Networking services like Virtual Private Cloud (VPC) are essential for creating isolated networks, managing communication between services, and enforcing security rules. Additionally, Identity and Access Management (IAM) is critical for defining roles and permissions, ensuring that only authorized users can access sensitive resources.

For this project, a specific set of GCP services was chosen to address the needs of a file-sharing application. Google Compute Engine was selected to handle the backend operations, providing virtual machines that support the server logic and manage user requests. This service allows the application to scale automatically in response to traffic demands, ensuring consistent performance. Google Cloud Storage was chosen as the main storage solution for user-uploaded files due to its ability to securely handle large volumes of data, with encryption and access controls in place to protect user information.

To manage networking and secure communication, Virtual Private Cloud (VPC) was implemented, allowing for the segmentation of resources and the configuration of firewall rules that limit access to critical components. A load balancer was also set up to distribute incoming traffic across multiple virtual machines, optimizing performance and minimizing downtime. IAM was utilized to control user permissions, ensuring that only authorized personnel have access to sensitive data and administrative functions.

These services were selected to provide a reliable, secure, and efficient infrastructure for the file-sharing application, leveraging GCP's capabilities to meet project goals.

Virtual Machines in Google Cloud

- **Machine Type:** I selected the **e2-micro** instance type to minimize costs, as it provides enough resources for a simple web server test.

The screenshot shows the Google Cloud console interface for creating a new VM instance. The 'Machine type' dropdown is open, displaying a list of options: e2-micro (0.25 vCPU, 1 GB memory), e2-small (0.5 vCPU, 2 GB memory), e2-medium (1 vCPU, 4 GB memory), and e2-high (2 vCPU, 8 GB memory). The 'e2-micro' option is selected. The 'Monthly estimate' on the right shows a total of \$3.44 for 2 vCPU and 1 GB memory. The 'Host error timeout' is set to 'Unspecified (default)'.

- **Operating System:** I chose Ubuntu 20.04 LTS due to its stability, wide support, and easy compatibility with web server packages like Apache and Nginx.
- **Region:** I selected **us-central1-c**.

The screenshot shows the Google Cloud console interface for the 'VM instances' page. The 'INSTANCES' tab is active, displaying a table with columns: Status, Name, Zone, Recommendations, In use by, Internal IP, External IP, and Connect. A single instance named 'test-instance' is listed in the 'us-central1-c' zone, with an internal IP of 10.128.0.2 and an external IP of 35.226.195.79.

- **Firewall:** Enabled HTTP and HTTPS traffic to allow web traffic to the server.

Firewalls

- ☒ Allow HTTP traffic
- ☒ Allow HTTPS traffic
- ☐ Allow Load Balancer Health checks

Network tags

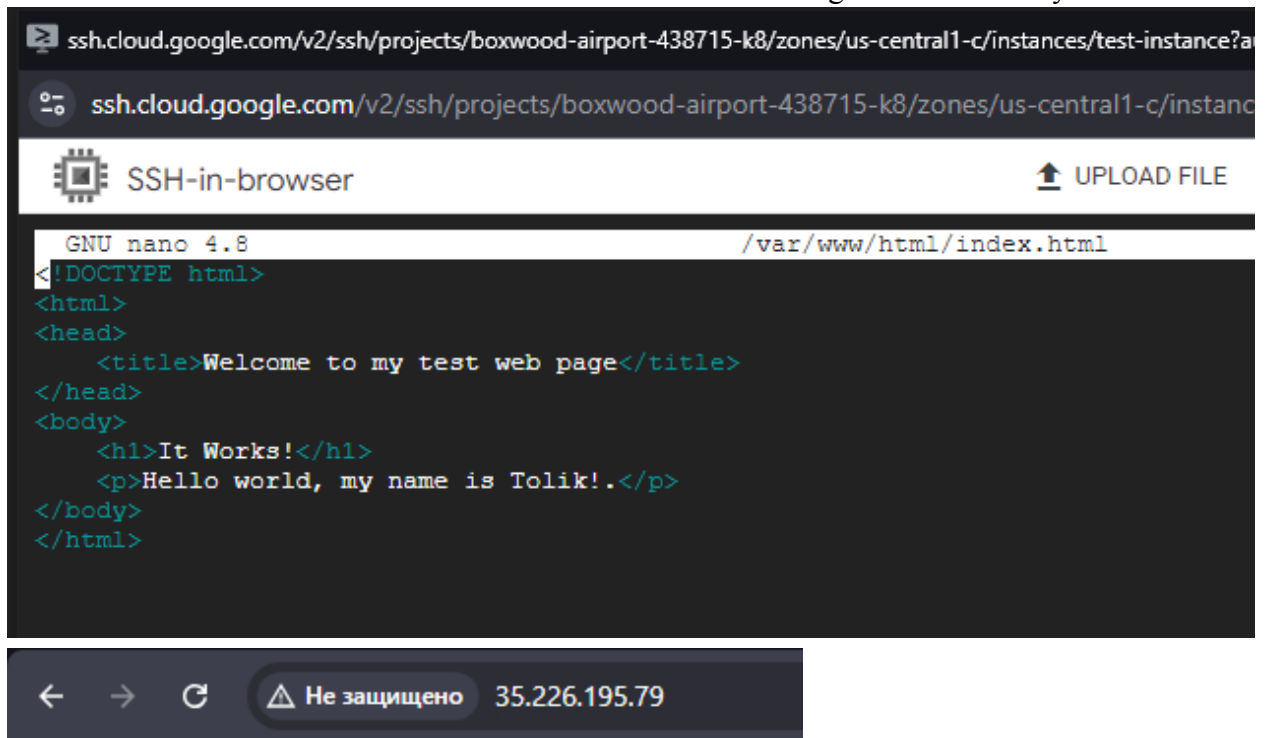
Network tags

- Connected via SSH using the Cloud Console's built-in SSH option, which allows easy access without needing external SSH clients.
- I chose Nginx because it's a widely-used web server with a simple setup process. Installed it using **apt** and verified the service was running.

```
ssh.cloud.google.com/v2/ssh/projects/boxwood-airport-438715-k8/zones/us-central1-c/instances/test-instance?authuser=0&hl=en_US&...
ssh.cloud.google.com/v2/ssh/projects/boxwood-airport-438715-k8/zones/us-central1-c/instances/test-instance?authuser=0&hl=en_US&...
SSH-in-browser
[+] UPLOAD FILE [v] DOWNLOAD FILE [v] [v] [v] [v]
Setting up nginx-common (1.18.0-0ubuntu1.6) ...
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /lib/systemd/system/nginx.service.
Setting up libjbig0:amd64 (2.1-3.1ubuntu0.20.04.1) ...
Setting up libnginx-mod-http-xslt-filter (1.18.0-0ubuntu1.6) ...
Setting up libwebp6:amd64 (0.6.1-2ubuntu0.20.04.3) ...
Setting up fonts-dejavu-core (2.37-1) ...
Setting up libjpeg-turbo8:amd64 (2.0.3-0ubuntu1.20.04.3) ...
Setting up libjpeg8:amd64 (8c-2ubuntu8) ...
Setting up libnginx-mod-mail (1.18.0-0ubuntu1.6) ...
Setting up fontconfig-config (2.13.1-2ubuntu3) ...
Setting up libnginx-mod-stream (1.18.0-0ubuntu1.6) ...
Setting up libtiff5:amd64 (4.1.0+git191117-2ubuntu0.20.04.14) ...
Setting up libfontconfig1:amd64 (2.13.1-2ubuntu3) ...
Setting up libgd3:amd64 (2.2.5-5.2ubuntu2.1) ...
Setting up libnginx-mod-http-image-filter (1.18.0-0ubuntu1.6) ...
Setting up nginx-core (1.18.0-0ubuntu1.6) ...
Setting up nginx (1.18.0-0ubuntu1.6) ...
Processing triggers for ufw (0.36-6ubuntu1.1) ...
Processing triggers for systemd (245.4-4ubuntu3.23) ...
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for libc-bin (2.31-0ubuntu9.16) ...
ernazartolegen@test-instance:~$ sudo systemctl start nginx
ernazartolegen@test-instance:~$ sudo systemctl start nginx
ernazartolegen@test-instance:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2024-10-17 14:52:02 UTC; 1min 39s ago
     Docs: man:nginx(8)
  Main PID: 45065 (nginx)
    Tasks: 3 (limit: 1134)
   Memory: 5.0M
   CGroup: /system.slice/nginx.service
           └─45065 nginx: master process /usr/sbin/nginx -g daemon on; master_process on;
             └─45066 nginx: worker process
               └─45067 nginx: worker process

Oct 17 14:52:02 test-instance systemd[1]: Starting A high performance web server and a reverse proxy server...
Oct 17 14:52:02 test-instance systemd[1]: Started A high performance web server and a reverse proxy server.
ernazartolegen@test-instance:~$
```

Created a basic HTML file to confirm the web server was serving content correctly.



It Works!

Hello world, my name is Tolik!.

Storage Solutions in Google Cloud

- **Bucket Name:** I named the bucket **ernazar-test-bucket** to ensure it was globally unique.

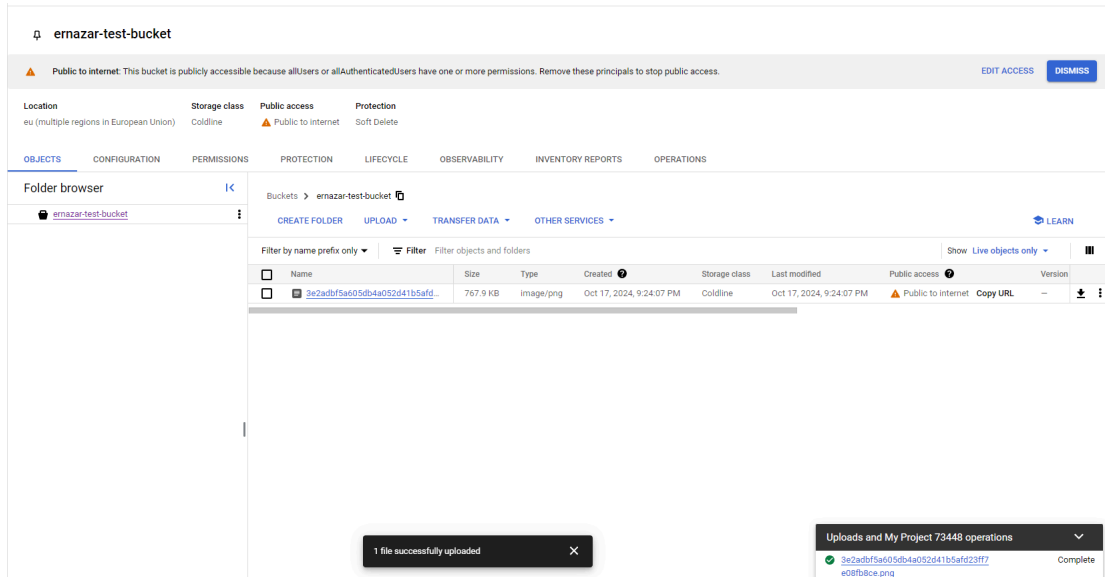
The screenshot shows the 'Create a bucket' wizard in Google Cloud. The left sidebar has 'Overview' (NEW), 'Buckets', 'Monitoring', and 'Settings'. The main area has five steps, all marked with a blue checkmark: 'Get Started' (Name: ernazar-test-bucket), 'Choose where to store your data' (Location: eu (multiple regions in European Union), Location type: Multi-region), 'Choose a storage class for your data' (Default storage class: Coldline), 'Choose how to control access to objects' (Public access prevention: On, Access control: Uniform), and 'Choose how to protect object data' (Soft delete policy: Default, Object versioning: Disabled, Bucket retention policy: Disabled, Object retention: Disabled, Encryption type: Google-managed). At the bottom are 'CREATE' and 'CANCEL' buttons. On the right, the 'Good to know' section includes 'Location pricing' with a table of storage rates and an 'ESTIMATE YOUR MONTHLY COST' section.

Item	Cost
eu (multiple regions in European Union)	\$0.007 per GB-month
With default replication	\$0.020 per GB written

- **Region:** I chose **eu** for better latency in my region.
- **Storage Class:** I selected **Coldline** as I don't need quick access to files and more cost-efficient. However, for quick access, **Standard** class might be more appropriate.
- **Access Control:** I chose **Uniform** control for the private access, but then added principle to **allUsers**.

The screenshot shows the 'Edit access to "ernazar-test-bucket"' dialog. It has a 'Principal' field with 'allUsers' and a 'Resource' field with 'ernazar-test-bucket'. Below is the 'Assign roles' section with a description: 'Roles are composed of sets of permissions and determine what the principal can do with this resource. [Learn more](#)'. There is a 'Role' dropdown menu set to 'Storage Legacy Object Reader' and an 'IAM condition (optional)' field with a '+ ADD IAM CONDITION' button. Below the role dropdown is a description: 'Grants permission to view objects and their metadata, excluding ACLs.' and a '+ ADD ANOTHER ROLE' button. At the bottom are 'SAVE', 'TEST CHANGES', and 'CANCEL' buttons.

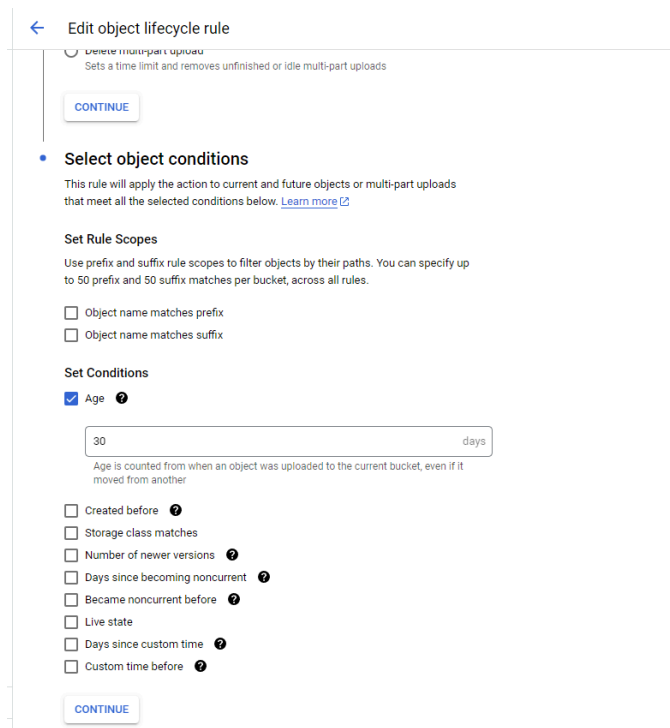
- Uploaded a sample image (sample.jpg) to test file storage. This could be useful for storing assets for a website, backups, or logs.



If you want to see what's inside, you can check it by following link below, but it will be accessible for only **30 days** from now, because of **lifecycle** rule:

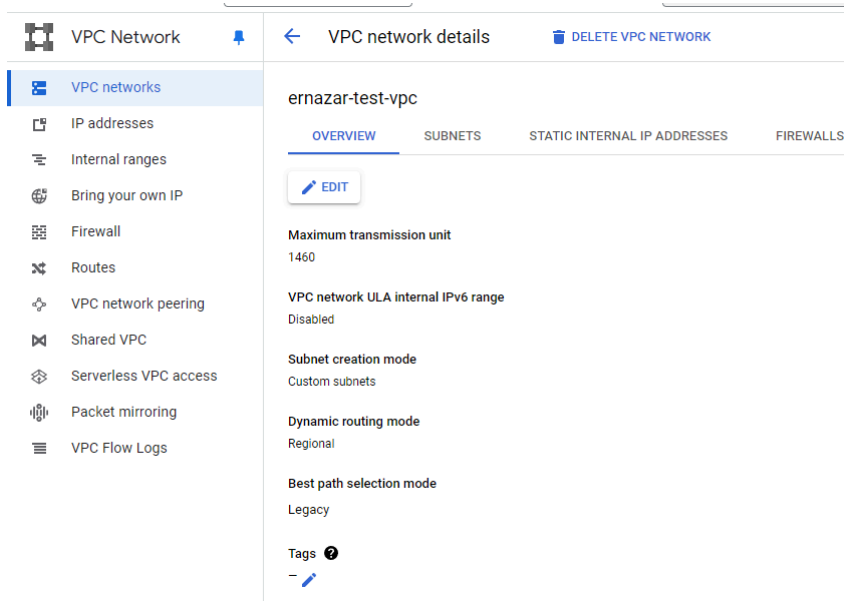
<https://storage.googleapis.com/ernazar-test-bucket/3e2adbf5a605db4a052d41b5afd23ff7e08fb8ce.png>

- Configured a lifecycle rule to automatically delete files older than 30 days. This can help manage storage costs by ensuring that old files that are no longer needed are automatically removed, particularly in use cases such as log files or backups.

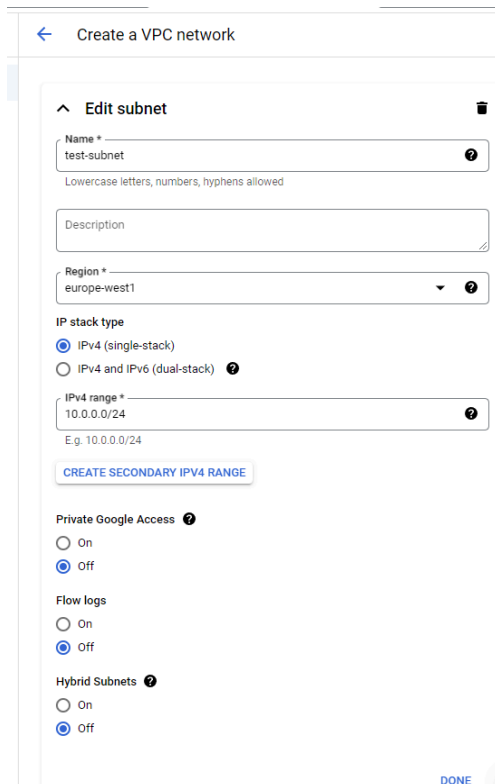


Networking in Google Cloud

- **VPC Name:** I named the VPC **ernazar-test-vpc** to easily identify it.



- **Subnets:** Created a subnet called **test-subnet** in the **europe-west1** region with the CIDR range 10.0.0.0/24. This allows for a flexible network architecture, and additional subnets can be added for future scaling or isolation of resources.



- **Use Case:** The VPC allows me to isolate my VM from other resources and networks in the cloud, ensuring security and performance.
- Created a firewall rule named **test-firewall** with SSH (TCP port 22) and HTTP (TCP port 80) traffic to the instances in the VPC.

The screenshot shows the Google Cloud Network Security console. The left sidebar lists various security services, with 'Firewall policies' selected. The main panel shows the configuration for a firewall rule named 'test-firewall'. The 'Source filter' is set to 'All instances in the network'. The 'Source IPv4 ranges' are set to '0.0.0.0/0'. The 'Second source filter' is set to 'None'. The 'Destination filter' is set to 'None'. The 'Protocols and ports' section is configured to allow 'TCP' traffic on ports '80,22'. The 'DISABLE RULE' checkbox is checked. The 'SAVE' button is visible at the bottom.

- **Importance:** Firewall rules are critical for securing cloud resources, allowing only necessary traffic while blocking unauthorized access.
- I tried to connect VM to VPC, but could not do it due to some restrictions in adding **new Network Interface** in my VM instance. I try to describe the actions without screenshots.

Network interfaces ?

Network interface is permanent

The screenshot shows the 'Network interfaces' page. A dropdown menu is open, showing the selected option 'default default IPv4 (10.128.0.0/20)'. Below the dropdown is a button labeled 'ADD A NETWORK INTERFACE'.

Cannot add an item

Firewalls

- Reconfigured the VM network interface to ensure it is connected to the newly created VPC (**ernazar-test-vpc**).
- Ping Google's DNS server (8.8.8.8) from the VM to verify internet connectivity through the VPC.
- **Verification:** The successful ping confirms that the VM is connected to the internet and can communicate with external servers.

Identity and Security Management

- **Identity and Access Management (IAM)** in Google Cloud Platform (GCP) allows for the precise control of access to resources. In this project, IAM was configured to manage user roles and permissions, ensuring that only authorized users could perform certain operations.
- **Creating Custom IAM Roles:** I created custom roles to ensure precise access control based on user responsibilities. For example, a custom role named "FileManager" was created with permissions like *storage.objects.create*, *storage.objects.get*, and *storage.objects.delete*. This allowed users to upload, download, and delete files without granting additional permissions.

IAM & Admin ← Create role

Custom roles let you group permissions and assign them to principals in your project or organization. You can manually select permissions or import permissions from another role. [Learn more](#)

Title *
FileManager
11 / 100 characters

Description
Created on: 2024-10-27
22 / 256 characters

ID *
FileManager

Role launch stage
Alpha

[+ ADD PERMISSIONS](#)

3 assigned permissions

Filter Enter property name or value

Permission	Status
☒ storage.objects.create	Supported
☒ storage.objects.delete	Supported
☒ storage.objects.get	Supported

Info Some permissions might be associated with and checked by third parties. These permissions contain the third party's service and domain name in the permission prefix.

[SHOW ADDED AND REMOVED PERMISSIONS](#)

[CREATE](#) [CANCEL](#)

- **Assigning IAM Roles to Users:** Specific roles were assigned based on user needs. Regular users were restricted to file operations, while administrators had more comprehensive permissions. This was done by navigating to **IAM & Admin > IAM**, and assigning roles through user emails or service accounts.
- **Data encryption At Rest:** Google Cloud Storage uses server-side encryption by default, ensuring that all stored files are encrypted without additional configuration.

- **Data encryption In Transit:** HTTPS was enforced for all data transmissions, securing communication between the server and users.
- **Access Control with IAM:** The storage bucket's default permissions were set to private. Only authenticated users with the necessary IAM roles could access files. File downloads were managed using signed URLs that expire after a specific time, providing secure, temporary access.

Testing and Quality Assurance

If I had access to Google Cloud Storage, I would use different types of tests to make sure the file-sharing application is reliable, secure, and performs well. These tests would look at how each part of the system works, how they interact together, and how the application handles lots of users at the same time.

Unit testing would focus on checking the individual parts of the code. For example, I would test functions for uploading and downloading files, user authentication, and database operations. To make sure the tests are consistent, I would use mock data to simulate Google Cloud Storage without using real data. This way, the testing environment would be safe and stable. I could use tools like **Mocha** for Node.js or **PyTest** for Python to automate these tests.

Integration testing would make sure that different parts of the system work well together. I would test how the backend communicates with Google Cloud Storage, making sure files upload, store, and download correctly. I would also check that permissions set through Identity and Access Management (IAM) are working as they should. For example, I might try uploading a file through the application interface, check that it's stored properly in Google Cloud, and then download it to make sure everything is working.

Load testing would help me see how the application performs under high traffic. Using tools like **Apache JMeter** or **Gatling**, I would simulate hundreds or thousands of users uploading and downloading files at the same time. The goal would be to check how well Google Cloud Storage and Compute Engine scale when lots of people use the app. I would look at metrics like how fast the system responds, how many errors happen, and how much CPU power is used. This would help me find any weak points and improve the app's performance.

Security testing would focus on finding any weaknesses in the system to make sure user data is safe. This would include trying to bypass access controls set through IAM, checking if data is encrypted both when stored and when sent, and making sure file downloads using signed URLs are secure. I would also do penetration tests to see how strong the security measures are against possible attacks.

Monitoring and Maintenance

To make sure the application performs well and stays stable over time, I would rely heavily on monitoring tools provided by Google Cloud. **Google Cloud Monitoring** would be my main tool for keeping track of the app's performance. It would help me monitor key metrics like CPU and memory usage, network activity, and storage capacity. I would set up custom dashboards to visualize this data in real time. This way, I could quickly see if something goes wrong and take action. For example, if the CPU usage suddenly spikes, I could investigate if there is an unexpected increase in traffic or a problem with the code.

Another useful feature in Google Cloud Monitoring is setting up alerts. I would create alerts that notify me by email or through a mobile app if certain thresholds are reached—like if the server response time gets too high or if the memory usage exceeds a safe limit. These alerts would help me identify potential problems before they affect users.

For tracking errors and troubleshooting, I would use **Google Cloud Logging**. This tool collects logs from the backend server, database, and other services, giving me detailed information on what's happening inside the application. If there's an error, I could look at the logs to understand what went wrong and fix it. I would also set up logging filters to focus on the most important issues, making it easier to manage the data.

Maintenance would involve regular software updates and database management. I would schedule updates to the backend software to ensure the latest security patches and features are installed. Regular maintenance tasks would also include checking storage usage, optimizing database queries for better performance, and cleaning up old or unused data to keep storage costs low. In case of emergencies, I would rely on automated backups, which are essential for restoring data quickly if something goes wrong.

Challenges and Solutions

Throughout the project, I faced several challenges that required careful problem-solving. One of the major issues I encountered was related to the **Virtual Private Cloud (VPC) and Virtual Machines (VMs)**. Initially, I struggled to connect the VPC to the VM due to limitations in the VM's configuration. The VM had certain restrictions that prevented it from properly linking with the custom network I set up in the VPC. This caused connectivity problems, as the VM couldn't access the secure network resources it needed for file management and other backend tasks.

To overcome this issue, I had to adjust the VM's settings and change the network configuration within Google Cloud Console. I experimented with different configurations for the network interfaces, ensuring they matched the IP address range and subnets defined in the VPC. After several attempts, I realized that the initial VM setup had a default network configuration that wasn't compatible with a custom VPC. I had to create a new VM from scratch, making sure to select the correct VPC and subnetwork during the creation process. This solution allowed me to establish a stable connection between the VPC and the new VM, and from that point on, the backend was able to communicate securely within the network.

Network configuration also presented challenges, especially with setting up firewall rules. In the beginning, the firewall rules were too restrictive, blocking legitimate traffic, which caused connectivity issues. I had to test and adjust these rules several times to strike the right balance between security and accessibility. This involved allowing only necessary ports for web traffic (like HTTP and HTTPS) while restricting administrative access to trusted IP addresses. Using Google Cloud VPC helped me create a secure network architecture, keeping the application safe from potential external threats.

Conclusion

Overall, working with Google Cloud Platform was a highly rewarding experience. The project allowed me to explore the different services offered by the platform, and I found it to be a reliable and flexible cloud environment. Setting up the file-sharing application was straightforward, and I appreciated how GCP services could be easily integrated to build a comprehensive system. From handling backend operations with Google Compute Engine to securely managing files in Google Cloud Storage, the platform provided a robust set of tools that helped me meet the project's requirements.

The security features were a key aspect of the project. Configuring Identity and Access Management (IAM) to control user permissions and ensure data security was an interesting challenge. Although it required several adjustments to get the permissions right, the end result was a system where each user role had appropriate access, and sensitive data remained protected. Additionally, tools like Cloud Monitoring and Cloud Logging made it easy to track the performance and health of the application, providing insights that helped me optimize and troubleshoot the system.

Looking ahead, I'm excited to continue working with Google Cloud Platform. I feel more confident in managing cloud infrastructure and understand the potential it has for handling both small-scale and large-scale projects. One of the services I'm eager to explore further is **BigQuery**, a data analytics platform that is exceptionally powerful for processing large datasets. In the context of future projects, I see it as a valuable tool for data-driven applications. BigQuery allows for running complex queries on massive datasets quickly, making it ideal for analyzing user behavior, identifying trends, and generating insights.

I am particularly interested in how BigQuery can integrate with **Data Studio**, Google's data visualization tool. Data Studio allows you to create interactive dashboards and reports based on BigQuery's analytics. This integration can make data analysis more accessible and visual, turning raw data into actionable insights. For future projects, I would love to use this combination to track key metrics of an application, visualize performance, and identify areas for improvement. Creating clear and interactive visualizations could also help in communicating data-driven results to stakeholders, making complex analytics understandable even to non-technical users.

My experience so far has focused on building infrastructure and ensuring the security of a web application, but working with BigQuery and Data Studio would bring a new layer of understanding—handling data analytics and visualization. I think using BigQuery to process large volumes of data, and then connecting it to Data Studio for real-time dashboards, could make future projects not only more insightful but also more engaging for end-users and clients.

References

Google Cloud. (n.d.). *Google Cloud documentation*. <https://cloud.google.com/docs>

Google Cloud. (n.d.). *Google Cloud Storage overview*. <https://cloud.google.com/storage>

Google Cloud. (n.d.). *Compute Engine documentation*. <https://cloud.google.com/compute>

Google Cloud. (n.d.). *Identity and Access Management (IAM) guide*.
<https://cloud.google.com/iam/docs>

Google Cloud. (n.d.). *Monitoring and logging in Google Cloud*.
<https://cloud.google.com/monitoring>

Appendices

Appendix A: Cloud service models differences

Cloud service model	Control	Flexibility	Use Cases	GCP Examples
IaaS	High control over infrastructure, OS, networking	Very flexible, users manage the infrastructure	Hosting websites, virtual machines, storage	Compute Engine, Cloud Storage
PaaS	Moderate control over applications and data	Moderate flexibility, developer-focused	Developing, testing, and deploying applications	App Engine, Cloud Functions
SaaS	Minimal control, mostly just using the software	Low flexibility, ready-to-use software	Using email, CRM, or collaboration tools	Google Workspace, Gmail